

RaPizz

Documentation Technique

Projet Java — Gestion de pizzeria avec livraison

Langage	Java 17+
Base de données	MySQL (via Docker)
IDE	Eclipse
Connecteur BDD	mysql-connector-j 9.6.0
Dépôt	github.com/LucasPokrywa/Projet-RaPizz

1. Présentation du projet

RaPizz est une application de bureau Java permettant la gestion complète d'une pizzeria avec service de livraison. Elle couvre la prise de commande client, l'affectation des livreurs et des véhicules, la gestion des paiements (avec solde client pré-approvisionné) ainsi qu'un tableau de bord analytique.

1.1 Objectifs fonctionnels

- Gérer les commandes clients (pizza, ingrédients, tarification)
- Attribuer automatiquement un véhicule disponible à chaque livreur
- Suivre l'activité des livreurs et des véhicules
- Débitier le compte client uniquement à la livraison
- Appliquer les règles métier : pizza offerte à la 10ème commande ou si livraison > 30 min
- Fournir un tableau de bord (chiffre d'affaires, meilleur client, etc.)

1.2 Règles métier clés

Deux cas déclenchent une pizza gratuite :

- Le client a commandé 10 pizzas (fidélité) — son compteur n'est PAS incrémenté dans ce cas
- Le délai de livraison dépasse 30 minutes — le compteur n'est PAS non plus incrémenté

Le débit du compte client s'effectue uniquement à la livraison (et non à la commande). Le solde est vérifié avant la préparation.

2. Architecture du projet

2.1 Structure des dossiers

Le dépôt est organisé en trois répertoires principaux :

Dossier	Contenu
RaPizz/	Projet Eclipse complet (sources, classes compilées, configuration)
sql/	Scripts de lancement et d'initialisation de la base de données
sql_schema/	Schémas de base de données et fichiers Looping (modélisation)

2.2 Packages Java (fr.rapizz)

Le code source est structuré selon le pattern DAO (Data Access Object) :

Package	Rôle
<code>fr.rapizz.dao</code>	Classes d'accès à la base de données (requêtes SQL, CRUD)
<code>fr.rapizz.model</code>	Classes métier / entités (Client, Pizza, Livreur, Véhicule, Commande...)
<code>fr.rapizz.view</code>	Interface utilisateur (fenêtres, formulaires, tableaux de bord)
<code>fr.rapizz.main</code>	Point d'entrée de l'application (App.java)

3. Base de données

La base de données est MySQL, lancée via Docker. Les scripts de démarrage et d'initialisation se trouvent dans le dossier sql .

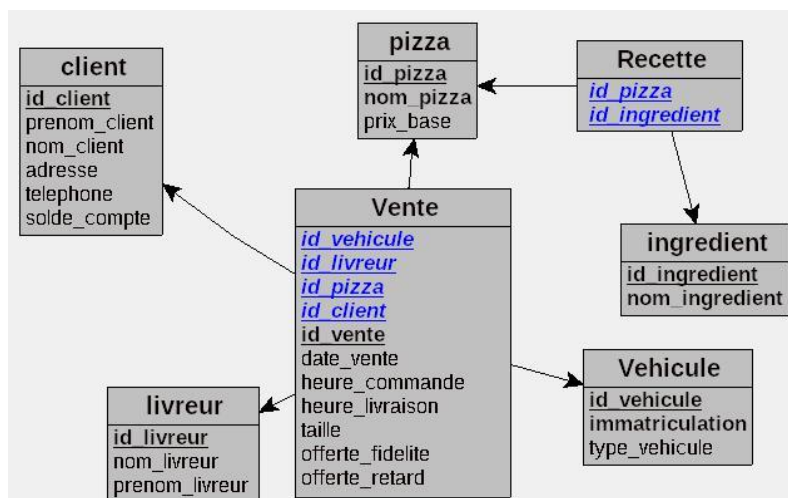
3.1 Entités principales

- Client — identifiant, solde, compteur de pizzas commandées
- Pizza — désignation, prix, ingrédients associés
- Ingrédient — nom, stock
- Livreur — identifiant, statistiques de retard
- Véhicule — identifiant, disponibilité
- Commande — date, heure de livraison effective, statut, montant

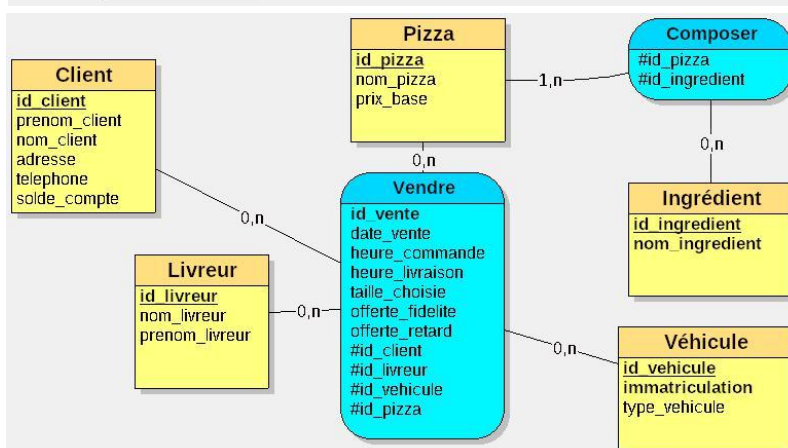
3.2 Schémas

Les fichiers Looping (modélisation MEA/MLD) et les schémas SQL sont disponibles dans sql_schema/. Le connecteur JDBC inclus est mysql-connector-j-9.6.0.jar (à la racine du projet).

MEA



MLD



3.3 Connexion

La connexion à la base est gérée par les classes du package dao. Le fichier JAR du connecteur MySQL doit être dans le classpath Eclipse.

| `mysql-connector-j-9.6.0.jar` ← à ajouter au Build Path Eclipse

4. Prérequis

Avant de lancer le projet, assurez-vous que les outils suivants sont installés et correctement configurés sur votre machine.

Outil	Version minimale	Utilisation
Java JDK	17+	Compilation et exécution de l'application
Docker	Toute version récente	Lancement du conteneur MySQL
Eclipse IDE	2021-09 ou +	Import et exécution du projet Java
Git	Toute version récente	Cloner le dépôt GitHub

4.1 Java JDK 17+

L'application est compilée et exécutée en Java. Le JDK 17 (LTS) est la version minimale requise.

- Téléchargement : <https://adoptium.net> ou <https://www.oracle.com/java/technologies/downloads/>
- Vérification de l'installation :

```
java -version
```

- La sortie doit afficher java version "17.x.x" ou supérieur
- Vérifier également que JAVA_HOME pointe vers le JDK (et non le JRE)

4.2 Docker

La base de données MySQL tourne dans un conteneur Docker. Docker Desktop est recommandé sous Windows et macOS.

- Téléchargement : <https://www.docker.com/products/docker-desktop>
- Vérification :

```
docker --version  
docker ps
```

- Docker doit être démarré avant de lancer les scripts BDD
- Aucune configuration manuelle de MySQL n'est nécessaire : tout est géré par les scripts

4.3 Eclipse IDE

Le projet est un projet Eclipse natif. L'IDE Eclipse est requis pour l'importer correctement.

- Téléchargement : <https://www.eclipse.org/downloads/> (choisir "Eclipse IDE for Java Developers")
- Version recommandée : 2021-09 ou plus récente
- Le connecteur MySQL (mysql-connector-j-9.6.0.jar) doit être ajouté au Build Path après import

4.4 Connecteur JDBC MySQL

Le fichier mysql-connector-j-9.6.0.jar est déjà inclus à la racine du dépôt. Il suffit de l'ajouter au classpath Eclipse :

- Clic droit sur le projet → Build Path → Configure Build Path
- Onglet Libraries → Add External JARs
- Sélectionner mysql-connector-j-9.6.0.jar à la racine du projet

4.5 Git (optionnel)

Pour cloner le dépôt en ligne de commande :

```
| git clone https://github.com/LucasPokrywa/Projet-RaPizz.git
```

Il est aussi possible de télécharger l'archive ZIP directement depuis GitHub (Code → Download ZIP).

5. Installation et démarrage

5.1 Démarrer la base de données

Se placer dans le dossier sql/ et exécuter le script adapté au système :

```
sql/script.bat      ← Windows  
sql/script.sh       ← Linux / macOS
```

Ces scripts lancent un conteneur Docker MySQL et initialisent le schéma et les données.

5.2 Importer et lancer le projet Eclipse

- Ouvrir Eclipse
- File → Import → Existing Projects into Workspace
- Sélectionner le dossier RaPizz/ du dépôt
- Cliquer sur Import Selected
- Vérifier que mysql-connector-j-9.6.0.jar est bien dans le Build Path
- Lancer App.java dans le package fr.rapizz.main

5. Fonctionnalités détaillées

5.1 Gestion des commandes

- Vérification de l'abonnement client (présence en BDD)
- Vérification du solde avant préparation
- Attribution automatique d'un véhicule disponible au livreur
- Calcul automatique de la gratuité (10ème pizza ou retard > 30 min)
- Débit du compte uniquement à la livraison

5.2 Suivi de la livraison

- Affectation d'un véhicule non utilisé à la prise de commande
- Enregistrement de l'heure de livraison effective
- Calcul du dépassement de délai (seuil : 30 minutes)

5.3 Tableau de bord analytique

- Chiffre d'affaires global
- Identification du meilleur client (volume commandé)
- Identification du plus mauvais livreur (nombre de retards)
- Véhicule le plus utilisé
- Pizza la plus et la moins demandée
- Ingrédient le plus consommé

6. Réponses aux questions du cahier des charges

Cette section répond point par point aux interrogations demandées dans le sujet « Gestion d'une entreprise de pizzas à domicile » (Pierre Lefebvre).

6.1 Menu — Carte des pizzas

Objectif : extraire le nom de chaque pizza, son prix (selon la taille) et ses ingrédients.

Règle de prix appliquée :

- Taille « naine » = $\text{prix_base} \times 2/3$
- Taille « humaine » = prix_base (prix de référence)
- Taille « ogresse » = $\text{prix_base} \times 4/3$

Requête SQL correspondante :

```
SELECT p.nom_pizza,
       p.prix_base * 2/3 AS prix_naine,
       p.prix_base      AS prix_humaine,
       p.prix_base * 4/3 AS prix_ogresse,
       GROUP_CONCAT(i.nom_ingredient ORDER BY i.nom_ingredient SEPARATOR
                     ', ')
       AS ingredients
FROM Pizza p
JOIN Composer c ON c.id_pizza = p.id_pizza
JOIN Ingredient i ON i.id_ingredient = c.id_ingredient
GROUP BY p.id_pizza, p.nom_pizza, p.prix_base
ORDER BY p.nom_pizza;
```

6.2 Fiche de livraison

Objectif : imprimer pour chaque commande le nom du livreur, le type de véhicule, le nom du client, la date, le retard éventuel, et le nom / prix de base de la pizza.

Requête SQL correspondante :

```
SELECT l.nom_livreur,
       v.type_vehicule,
       c.nom_client,
       cmd.date_commande,
       p.nom_pizza,
       p.prix_base,
       TIMESTAMPDIFF(MINUTE, cmd.date_commande,
                     cmd.date_livraison) AS duree_livraison_min,
       CASE
         WHEN TIMESTAMPDIFF(MINUTE, cmd.date_commande,
                             cmd.date_livraison) > 30
```

```

        THEN 'OUI' ELSE 'NON'
    END AS retard
FROM Commande cmd
JOIN Client c ON c.id_client = cmd.id_client
JOIN Livreur l ON l.id_livreur = cmd.id_livreur
JOIN Vehicule v ON v.id_vehicule = cmd.id_vehicule
JOIN Pizza p ON p.id_pizza = cmd.id_pizza
ORDER BY cmd.date_commande DESC;

```

6.3 Questions diverses

Véhicules n'ayant jamais servi

Requête SQL :

```

SELECT v.id_vehicule, v.type_vehicule
FROM Vehicule v
WHERE v.id_vehicule NOT IN (
    SELECT DISTINCT id_vehicule FROM Commande
);

```

Nombre de commandes par client

Requête SQL :

```

SELECT c.nom_client,
       COUNT(cmd.id_commande) AS nb_commandes
FROM Client c
LEFT JOIN Commande cmd ON cmd.id_client = c.id_client
GROUP BY c.id_client, c.nom_client
ORDER BY nb_commandes DESC;

```

Moyenne des commandes par client

Requête SQL :

```

SELECT AVG(nb_commandes) AS moyenne_commandes
FROM (
    SELECT COUNT(id_commande) AS nb_commandes
    FROM Commande
    GROUP BY id_client
) AS sous_requete;

```

Clients ayant commandé plus que la moyenne

Requête SQL :

```

SELECT c.nom_client,
       COUNT(cmd.id_commande) AS nb_commandes
FROM Client c
JOIN Commande cmd ON cmd.id_client = c.id_client
GROUP BY c.id_client, c.nom_client
HAVING nb_commandes > (

```

```

SELECT AVG(nb_cmd) FROM (
    SELECT COUNT(id_commande) AS nb_cmd
    FROM Commande GROUP BY id_client
) AS moy
)
ORDER BY nb_commandes DESC;

```

6.4 Statistiques d'activité

Meilleur client (chiffre d'affaires généré)

```

SELECT c.nom_client,
       SUM(cmd.montant) AS total_depense
FROM Client c
JOIN Commande cmd ON cmd.id_client = c.id_client
WHERE cmd.gratuite = FALSE
GROUP BY c.id_client, c.nom_client
ORDER BY total_depense DESC
LIMIT 1;

```

Plus mauvais livreur (nombre de retards)

```

SELECT l.nom_livreur,
       COUNT(*) AS nb_retards
FROM Livreur l
JOIN Commande cmd ON cmd.id_livreur = l.id_livreur
WHERE TIMESTAMPDIFF(MINUTE, cmd.date_commande,
                    cmd.date_livraison) > 30
GROUP BY l.id_livreur, l.nom_livreur
ORDER BY nb_retards DESC
LIMIT 1;

```

Pizza la plus demandée / la moins demandée

```

-- La plus demandée
SELECT p.nom_pizza, COUNT(*) AS nb_ventes
FROM Pizza p
JOIN Commande cmd ON cmd.id_pizza = p.id_pizza
GROUP BY p.id_pizza, p.nom_pizza
ORDER BY nb_ventes DESC LIMIT 1;

```

```

-- La moins demandée
SELECT p.nom_pizza, COUNT(*) AS nb_ventes
FROM Pizza p
JOIN Commande cmd ON cmd.id_pizza = p.id_pizza
GROUP BY p.id_pizza, p.nom_pizza
ORDER BY nb_ventes ASC LIMIT 1;

```

Ingrédient le plus consommé

```
SELECT i.nom_ingredient,  
       COUNT(*) AS nb_fois_utilise  
FROM Ingredient i  
JOIN Composer c   ON c.id_ingredient = i.id_ingredient  
JOIN Pizza p      ON p.id_pizza      = c.id_pizza  
JOIN Commande cmd ON cmd.id_pizza    = p.id_pizza  
GROUP BY i.id_ingredient, i.nom_ingredient  
ORDER BY nb_fois_utilise DESC  
LIMIT 1;
```

6.5 Règles de gestion implémentées en Java / JDBC

Le cahier des charges demande d'exploiter les triggers, procédures stockées et l'API JDBC. Voici la correspondance avec le code du projet :

Règle métier	Implémentation
Vérification du solde	Requête SELECT avant INSERT en JDBC (package dao)
Gratuit à la 10ème pizza	Compteur client incrémenté côté Java ; gratuite=TRUE si = 10
Gratuit si retard > 30 min	Calcul TIMESTAMPDIFF au moment de la livraison (trigger ou DAO)
Non-incrémentation du compteur si gratuit	Condition Java : si gratuite=TRUE, pas d'UPDATE du compteur
Débit uniquement à la livraison	UPDATE solde déclenché par l'action "Livraison confirmée"
Refus si solde insuffisant	Exception levée côté Java, commande non insérée

7. Contributeurs

Le projet est développé par 4 contributeurs (voir onglet Contributors sur GitHub). Dépôt public, sans release publiée à ce jour.

Lien du dépôt :

| <https://github.com/LucasPokrywa/Projet-RaPizz>